

Trilha de Node.js

Módulo 07: Aplicações Avançadas

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 7: Aplicações Avançadas com Node.js

Objetivo:

Explorar funcionalidades avançadas do Node.js para capacitar os alunos a desenvolver aplicações modernas e escaláveis com alta performance.

7.1: Programação assíncrona com Promises e async/await

O que é Programação Assíncrona?

- Permite que o código execute tarefas sem bloquear o fluxo principal.
- Torna as aplicações mais eficientes ao lidar com I/O.

Promises

1. Representam a eventual conclusão (ou falha) de uma operação assíncrona.

Exemplo:

```
const promessa = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Sucesso!'), 1000);
});

promessa.then(console.log).catch(console.error);
```

Async/Await

1. Sintaxe mais simples e legível para lidar com promises.
2. Exemplo:

```
async function executar() {
  try {
    const resultado = await promessa;
    console.log(resultado);
  } catch (erro) {
    console.error(erro);
  }
}

executar();
```

7.2: Trabalhando com worker_threads para multitarefa

O que é Worker Threads?

- Permite executar código JavaScript em threads separados.
- Ideal para tarefas pesadas que não devem bloquear o Event Loop.

Exemplo Simples

- Arquivo principal (**index.js**)

```
const { Worker } = require('worker_threads');
const worker = new Worker('./worker.js');

worker.on('message', (mensagem) =>
  console.log('Mensagem do Worker:', mensagem)
);
```

- Arquivo do Worker (**worker.js**)

```
const { parentPort } = require('worker_threads');

parentPort.postMessage('Tarefa concluída no Worker!');
```

7.3: Integração de Websockets com Socket.io

O que é Socket.io?

- Uma biblioteca que facilita a comunicação bidirecional em tempo real entre cliente e servidor.

Instalação

```
npm install socket.io
```

Exemplo Básico

1. Servidor (**server.js**)

```
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server);

io.on('connection', (socket) => {
  console.log('Usuário conectado:', socket.id);
  socket.on('mensagem', (msg) => console.log('Mensagem recebida:', msg));
});

server.listen(3000, () => console.log('Servidor rodando na porta 3000'));
```

2. Cliente (HTML)

```
<script src="/socket.io/socket.io.js"></script>
<script>
  const socket = io();
  socket.emit('mensagem', 'Olá do cliente!');
</script>
```

7.4: Estratégias de escalabilidade com clusters e child processes

Clusters

1. Permitem aproveitar todos os núcleos do processador para melhorar a performance.

Exemplo de Cluster

```
const cluster = require('cluster');
const http = require('http');
const numCPUs = require('os').cpus().length;

if (cluster.isMaster) {
  console.log(`Processo mestre: ${process.pid}`);
  for (let i = 0; i < numCPUs; i++) {
    cluster.fork();
  }
} else {
  http.createServer((req, res) => res.end('Cluster ativo')).listen(8000);
}
```

Child Processes

1. Permite criar processos filhos para tarefas independentes.

Exemplo de Child Process

```
const { spawn } = require('child_process');

const processo = spawn('ls', ['-lh']);
processo.stdout.on('data', (data) => console.log(`Saída: ${data}`));
```

7.5: Monitoramento e logging com PM2 e Winston

PM2

- Iniciar um processo:

```
pm2 start app.js
```

Winston

- Biblioteca para registro de logs estruturados.

Exemplo com Winston

```
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'app.log' })
  ]
});

logger.info('Informação registrada no log!');
```

Lista de Exercícios

Questões Teóricas

- Explique a diferença entre callbacks, promises e async/await.

- O que é o Worker Threads no Node.js e qual sua utilidade?
- Como o Socket.io facilita a comunicação em tempo real?
- Qual é o papel dos clusters em aplicações Node.js?
- Liste três vantagens do PM2 para o gerenciamento de processos.
- O que é um child process e como ele é utilizado?
- Explique o conceito de logging estruturado e sua importância.
- Qual é a vantagem de usar o Winston para logs?
- Por que é importante escalar aplicações Node.js em ambientes de produção?
- Como a programação assíncrona melhora a performance de aplicações?

Questões Práticas

- Implemente uma função que utilize async/await para buscar dados de uma API.
- Configure um Worker Thread que execute uma tarefa independente.
- Crie um servidor Socket.io que receba mensagens de um cliente.
- Implemente um cluster que utilize todos os núcleos do processador.
- Configure um processo filho que execute um comando do sistema operacional.
- Instale o PM2 e monitore uma aplicação Node.js.
- Configure um logger com Winston para salvar logs em arquivo.
- Crie uma aplicação que registre logs de erros e informações.
- Use o Socket.io para criar um chat em tempo real.
- Combine Worker Threads e logging para monitorar tarefas em segundo plano.